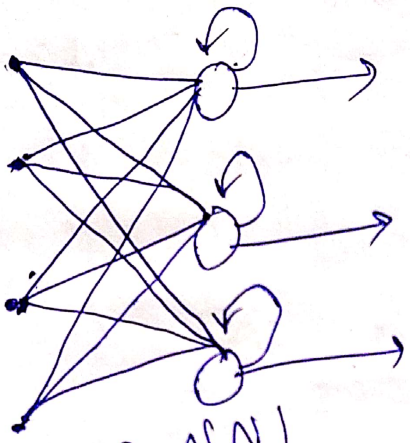
CO-3

Recurrent Neural Networks

* RNN's allow the network to remember past information by feeding the output from one step to the next step.
* RNN's use loops that allow information from previous steps to be fed back into the network.



(RNN).

Components

I) Recurrent Neurons

* The fundamental processing unit in RNN is a Recurrent unit.
* Recurrent units can remember information from prior steps by feeding back their hidden state.

Scanned with OKEN Scanner

RNN Unfolding

* It is the process of expanding the recurrent structure over time steps.

I. Hidden State Calculation.

$$h_t = \sigma(Ux_t + Wh_{t-1} + B)$$

h_t = Current hidden state.
x_t = Current input
h_{t-1} = Previous hidden state.
U = Input weight matrix
W = Recurrent weight matrix
B = Bias vector
σ = Activation function.

Types of RNN's

1) One-to-one RNN ⇒ Single input & Single output
2) One-to-many RNN ⇒ Single input, multiple outputs.
3) Many-to-one RNN ⇒ Multiple inputs & Single output
4) Many-to-Many RNN ⇒ Sequence of inputs generate sequence of outputs

Scanned with OKEN Scanner

Applications

→ Language Modelling & Generating Text
→ Speech Recognition
→ Machine Translation.
→ Time series prediction.

Vanishing Gradient

* This is a challenge that emerges during backpropagation when the derivatives or slopes of the activation functions become progressively smaller as we move backward through the layers of a neural network.

Vanishing & Exploding gradients

→ Batch Normalization.
→ Activation function.
→ Skip Connections & Residual Networks
→ Long Short-Term Memory Networks (LSTM's) & Gated Recurrent Units (GRU's).
→ Gradient Clipping.

* The exploding gradient problem is a challenge encountered during training deep neural networks. It occurs when the gradients of the network's

Scanned with OKEN Scanner

loss function with respect to the weights become excessively large.

→ Vanishing ⇒ $\left\| \frac{\partial h_i}{\partial h_{i-1}} \right\| < 1$

→ Exploding ⇒ $\left\| \frac{\partial h_i}{\partial h_{i-1}} \right\|_2 > 1$

Truncated BPTT

* Truncated Backpropagation Through Time is a technique used to train RNN's efficiently, particularly for long sequences.

→ Computational Efficiency.

→ Mitigating Vanishing/Exploding Gradients.

→ Enabling Online Learning.

Ex:- V Gradient ⇒ $W_h = 0.5$.

E Gradient ⇒ $W_h = 2$.

$$\frac{\partial L}{\partial h_0} = \frac{\partial L}{\partial h_T} \prod_{t=1}^{T} (W_h \times \tanh'(z_t))$$

$$\frac{\partial L}{\partial h_T} = 1$$

Scanned with OKEN Scanner

Q. $W_h = 0.5$

$\tanh'(z) = 0.5$

$W_h \times \tanh'(z) = 0.5 \times 0.5$

$= 0.25$

$$\frac{\partial L}{\partial h_0} = 1 \times (0.25)^5$$

$0.25^2 = 0.0625$

$0.25^3 = 0.015625$

$0.25^4 = 0.00390625$

$0.25^5 = \boxed{0.0009765625} \Rightarrow \frac{\partial L}{\partial h_0}$

↓

Almost zero, where it becomes vanishing gradient.

Q. Exploding Gradient

$W_h = 2$

$\tanh'(z) = 0.5$

$2 \times 0.5 = 1$

$$\frac{\partial L}{\partial h_0} = 1^5 = 1$$

Loss functions in RNN's

* Used to minimize the loss during training.

Types

→ Mean Squared Error

$$MSE = \frac{1}{n} \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$

→ Cross - Entropy Loss

$$L = -\frac{1}{N} \sum_{i=1}^{n} [y_i \log(\hat{y}_i) + (1-y_i) \log(1-\hat{y}_i)]$$

→ Negative Log Likelihood (NLL)

$$L = -\log(P(y|x))$$

$$L = -\sum y_i \log(\hat{y}_i)$$

→ Categorical Cross - Entropy

$$L = -\sum_{i=1}^{C} y_i \log(\hat{y}_i)$$

→ Hinge Loss

$$L = \max(0, 1 - y\hat{y})$$

$$y \in \{-1, 1\}$$

→ Connectionist Temporal Classification (CTC) Loss.

$$L = -\log(P(Y|X))$$

Scanned with OKEN Scanner

→ Sequence Loss

$$L = -\sum_{t=1}^{T} \log P(y_t | x_t)$$

→ Reinforcement learning - based Loss.

$$L = -E[\log \pi(a|s) R]$$

Long Short Term Memory (LSTM)

* Type of RNN's that can detain long-term dependencies in sequential data such as time series, text & speech.

* Uses memory cell and gates to control the flow of information.

Three types of gates in an LSTM

1) input gate = controls flow of info. the memory cell.
2) forget gate = Controls the flow of info out of the memory cell.
3) output gate = Controls the flow of info out of the LSTM & into the output.

Gated Recurrent Units (GRU's)

* To use gating mechanisms to selectively update the hidden state at each time step allowing them to remember important information while discarding irrelevant details.

→ Update gate

→ Reset gate

Key Con

1) Forget

$f_t =$

2) Inpu

i t

3) Cell

4) Outp

to

C

Scanned with OKEN Scanner

→ Update gate ⇒ This decides how much information from previous hidden state should be retained for the next time step.

→ Reset gate ⇒ This determines how much of the past hidden state should be forgotten.

Key Components

1) Forget Gate ⇒ Decides which info to discard from the previous state.

$$f_t = \sigma(W_f [h_{t-1}, x_t] + b_f)$$

2) Input Gate ⇒ Determines what new info to store in the cell state.

$$i_t = \sigma(W_i [h_{t-1}, x_t] + b_i)$$

3) Cell State (C_t) ⇒ The memory unit that stores relevant information over time.

$$C_t = f_t * C_{t-1} + i_t * C_t$$

4) Output Gate (O_t) ⇒ Decides which info to send to the next hidden state.

$$O_t = \sigma(W_o [h_{t-1}, x_t] + b_o)$$

Scanned with OKEN Scanner

5) Hidden State (h_t)

* The final output passed to the next time step.

$$h_t = O_t * \tanh(C_t)$$

Attention models (Attention mechanisms)

* These are used to provide an additional focus on a specific component.
* Reduces larger, more complicated tasks into smaller.
* Helps the model to focus on most relevant parts.

Use Cases

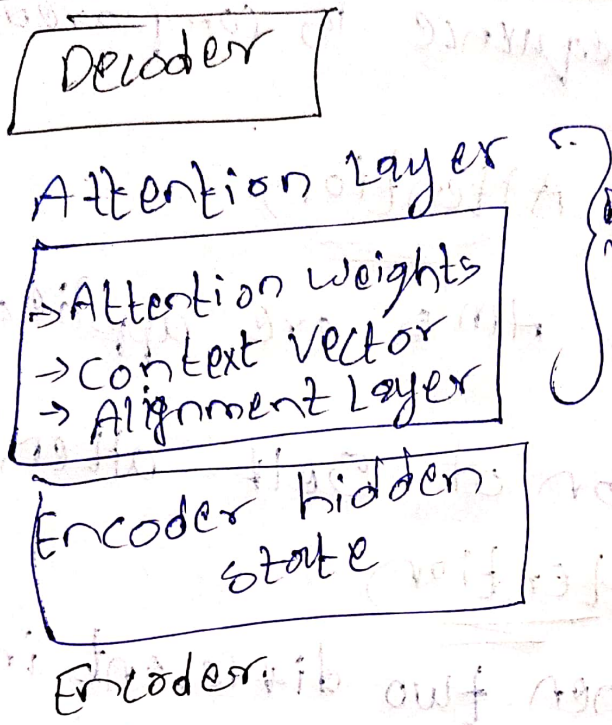
→ Image Classification.
→ Image generation.
→ Object Detection.
→ Image Synthesis.

Workflow

1. Breaking Down the input
2. Picking out important bits.
3. Assigning Importance.
4) Focusing Attention.
5. Putting It All together.

Scanned with OKEN Scanner

Architecture



Types

1) Soft Attention

* It assigns a weight to each input part representing its importance for that task.

$$Attention(Q, K, V) = Softmax\left(\frac{QK^T}{\sqrt{d_K}}\right)V$$

Q: Query, K: key, V: value, d_K: dimensionality

2) Hard Attention.

* Hard attention makes a binary decision, ie it selects specific parts of the input to focus on.

Scanned with OKEN Scanner

3) Self Attention (Intra-Attention)

* Input sequence is compared to itself.

4) Multi-head Attention

* Multiple AM's are applied in parallel.
* Extension of self-attention.

5) Cross-Attention

* Used when two different input sequences are compared.

6) Scaled Dot-Product Attention

* Computes the attention scores using the dot product of query & key vectors to avoid excessively large values in the dot product.

7) Location-Based Attention

* Use the position or location of the input elements as part of the attention calculation.

8) Global Attention.

Deep Dream

* This progra
neural ne
image p
Pareidoli

* The netwo
detect
appear
Net. data

Deep Art

* This a
given
your

Scanned with OKEN Scanner

Deep Dream

* This program uses a convolutional neural network to find & enhance image patterns using algorithmic Pareidolia.

* The network has been trained to detect certain patterns which appear frequently in the Image-Net data.

Deep Art

* This adapts the style from any given painting and applies it to your picture or video.

Scanned with OKEN Scanner